

STIX Lab — From an Investigation to a Shared Threat Graph

Day 3 · from beginner to advanced · you turn OSINT findings into a STIX 2.1 knowledge graph, then automate and share it

By the end you can read STIX, build a valid bundle in Python, convert raw OSINT into STIX automatically with parallel agents, and explore it on a team dashboard — the way OpenCTI, MISP and a modern CTI team work.

STIX (Structured Threat Information eXpression) is an OASIS open standard (v2.1) for describing intelligence as a machine-readable **graph**: entities (SDOs), technical observables (SCOs) and typed relationships (SROs), with sharing rules (TLP) and confidence. **TAXII** is the protocol that moves STIX between tools.

Learning objectives

- Read a STIX bundle and name every object type (SDO / SCO / SRO), its TLP marking and confidence.
- Build a valid STIX 2.1 bundle in Python with the `stix2` library.
- Map an Admiralty-rated OSINT finding (Day 2) onto STIX objects.
- Automate OSINT → STIX and run **parallel collector agents** that converge into one bundle.
- Visualise and share the graph; understand where OpenCTI / MISP / MITRE ATT&CK / n8n fit.

Setup

```
cd "Day_3/STIX Lab"
pip install stix2
```

Files: `backend/` (the scripts), `frontend/` (the team dashboard), `data/` (bundles).

Part 1 — Read STIX (beginner, no code, 30 min)

Open `data/bundle.json` (generate it first with `python3 backend/build_bundle.py`) and `frontend/index.html` in a browser.

The three families:

Family	What it is	Examples in the bundle
SDO — Domain Objects	the entities	<code>identity</code> , <code>threat-actor</code> , <code>malware</code> , <code>attack-pattern</code> , <code>indicator</code> , <code>report</code> , <code>note</code> , <code>opinion</code>
SCO — Cyber-observables	the technical facts	<code>domain-name</code> , <code>ipv4-addr</code>
SRO — Relationship Objects	the links	<code>relationship</code> (uses, targets, indicates, communicates-with), <code>sighting</code>

Every object carries an `id` (type--UUID), `created/modified`, often `created_by_ref`, a `confidence` (0–100) and `object_marking_refs` pointing to a **TLP** marking.

Questions

- 1.1 List each object in `bundle.json` and put it in the right family (SDO / SCO / SRO).
- 1.2 Find the `indicator`. What is its `pattern`? What does the pattern language express?
- 1.3 Find the TLP marking. What does `TLP:AMBER` allow you to do with this bundle?
- 1.4 The `opinion` object encodes the analyst's judgement. Re-express it on the **Admiralty scale** (source letter + info digit). Which STIX fields carry provenance and confidence?
- 1.5 On the dashboard, follow the edges from `UNC-Falcon`. In one sentence, what is the story the graph tells?

Part 2 — Build a bundle in Python (intermediate, 40 min)

Read `backend/build_bundle.py`. It creates the same case with the `stix2` library.

```
python3 backend/build_bundle.py      # writes data/bundle.json + frontend/data.js
```

Tasks

- 2.1 A STIX object is immutable and typed. Show the three lines that create the `Indicator` and explain each argument (`pattern`, `pattern_type`, `valid_from`, `confidence`).
- 2.2 Add a **second indicator** for a new phishing domain `login-aeroparts.net`, plus a `relationship` "indicates" from it to the threat actor. Re-run and confirm it appears on the dashboard.
- 2.3 Add a `Sighting` for your new indicator (`count`, `where_sighted_refs` = the victim). What does a Sighting record that an Indicator does not?

2.4 Attach the ATT&CK technique **T1566.001 (Spearphishing Attachment)** as a second `attack-pattern` with an `external_reference` to MITRE ATT&CK. Why is the `external_id` the important field?

Part 3 — Automate & scale (advanced, 60 min)

3a. OSINT findings → STIX

`backend/osint_to_stix.py` converts a plain findings file into a bundle.

```
cat data/findings.json
python3 backend/osint_to_stix.py data/findings.json    # -> data/osint_bundle.json
```

3.1 Read the mapping: one `Indicator` per domain, `SCOs` for domains/IPs, one `ObservedData`, one `Report`. Why is `Report` the right object to group an investigation?

3.2 Extend the input schema to also carry `emails`, and emit an `email-addr` `SCO` for each. Re-run and inspect.

3b. Parallel, agentic collection

`backend/agents.py` runs several **collector agents concurrently**, then a writer agent merges them into one bundle — the shape of an OpenCTI/n8n pipeline.

```
python3 backend/agents.py    # 3 agents run in parallel -> data/agentic_bundle.json
```

3.3 The run reports its wall-clock time. Explain why it is close to the **slowest** agent, not the **sum** of the agents. Which Python construct makes that happen?

3.4 Add a fourth agent `agent_certs()` that returns a `certs` list, and teach the writer to emit an `x509-certificate` `SCO`. Keep the merge idempotent.

3.5 Point `frontend/data.js` at `agentic_bundle.json` (or copy it) and view the agentic graph on the dashboard.

3c. Orchestrate & share (design)

- Open `backend/n8n_workflow.json`. It mirrors `agents.py` as an **n8n** workflow: a weekly trigger fans out to two HTTP "agents" (`crt.sh`, `news`) that converge on a writer node, which loads the bundle into **OpenCTI** via **TAXII**. Import it into n8n (Workflows → Import from File) and read the node graph.
- **OpenCTI** and **MISP** are the team platforms that store and share STIX; **MITRE ATT&CK** is itself published as STIX; **TAXII** is the transport.

3.6 In two or three sentences, describe your target-state pipeline: which agents run in parallel, what each emits, where the STIX lands, and who consumes it.

Part 4 — Go further: validate, rate, monitor, enrich, share (expert, 45 min)

This is where a bundle becomes production CTI.

4a. Validate strictly

```
python3 backend/validate.py data/bundle.json      # stix2 strict parse (spec-enforced)
stix2_validator data/bundle.json                  # OASIS schema + best-practice checks
```

4.1 What does `stix2.parse(strict)` guarantee that a JSON schema check does not, and vice-versa? Why run both?

4b. Admiralty <-> STIX, programmatically

`backend/admiralty.py` converts a Day 2 rating into a STIX `confidence` and an `Opinion`.

```
python3 backend/admiralty.py      # A1->95, B2->80, ... and back to a band
```

4.2 Use `rating_to_opinion("B2", indicator.id)` in `build_bundle.py` instead of the hard-coded `Opinion`. Re-validate. Why is `min(source, info)` the cautious mapping?

4c. Monitor for change (idempotent)

A monitor reports only what is new. `backend/diff_monitor.py` diffs two runs.

```
python3 backend/diff_monitor.py data/bundle.json data/agentive_bundle.json
```

4.3 Run the agentive pipeline twice and diff the two outputs. Why should a second run with the same inputs produce **no** additions? What breaks idempotence, and how do you fix it (natural keys / deterministic ids)?

4d. Enrich from MITRE ATT&CK (live)

ATT&CK is published as STIX. `backend/enrich_attack.py` pulls a technique's authoritative record.

```
python3 backend/enrich_attack.py T1566.002      # needs network; cache the bundle in class
```

4.4 Enrich the two ATT&CK techniques in your bundle. Add the fetched `description` to a `Note` linked to each `attack-pattern`. Why reference by `external_id` rather than copying the text?

4e. Share it: OpenCTI & TAXII

`backend/push_opencti.py` shows the last hop: `import_bundle_from_json` into OpenCTI (idempotent upsert), or POST to a TAXII 2.1 collection. Read it.

4.5 Draw the full production pipeline in five boxes: collectors (parallel) -> writer (STIX) -> validate -> OpenCTI/TAXII -> team + alerts. Where do TLP and confidence enforce themselves along the way?

Deliverables

One zip `STIX_<team>.zip` with:

- 1. Your extended `build_bundle.py` (second indicator + sighting + ATT&CK pattern) and the regenerated `bundle.json`.
- 2. `osint_bundle.json` and `agentic_bundle.json` from your runs, plus your fourth agent in `agents.py`.
- 3. Answers to the numbered questions (1.1 → 3.6).
- 4. One screenshot of the team dashboard showing your extended graph.
- 5. (Bonus) your Admiralty-wired `build_bundle.py`, a diff-monitor run, and ATT&CK-enriched notes.

Grading grid (100 pts)

Item	Pts	What earns the points
Read STIX (Part 1)	20	correct family classification, TLP, Admiralty mapping
Build a valid bundle (Part 2)	25	new indicator + relationship + sighting render; ATT&CK external ref correct
OSINT → STIX (3a)	15	email SCO added, valid bundle
Agentic pipeline (3b)	25	fourth agent + x509 SCO, concurrency explained, idempotent merge
Orchestration & sharing (3c)	10	coherent target-state pipeline; roles of OpenCTI/MISP/TAXII/ATT&CK
Go further (Part 4)	+bonus	validation, Admiralty mapping, idempotent diff, ATT&CK enrichment, share path
Deliverable quality	5	reproducible, screenshot, no invalid STIX

Optional Extensions (Bonus)

- **Validate** every bundle with the official `stix2-validator` and fix any warning.

- **Diff two runs:** detect new objects between last week's and this week's bundle and raise an alert (the Day 1 pipeline discipline: idempotent, timestamped).
- **Confidence as Admiralty:** write a helper that turns a `Bx` rating into a `confidence` number and an `Opinion`, consistently.
- **Real connectors:** replace the stubbed agents with real `crt.sh` + passive DNS + news calls (respect rate limits, TLP the outputs).
- **Push to OpenCTI:** stand up OpenCTI locally and load a bundle with `pycti`.